CACS402: Cloud computing
BCA 7th Semester

unit-6 Cloud Platforms and Applications

Mechi Multiple Campus

Bhadrapur

Cloud Platforms and Applications

Cloud Platforms

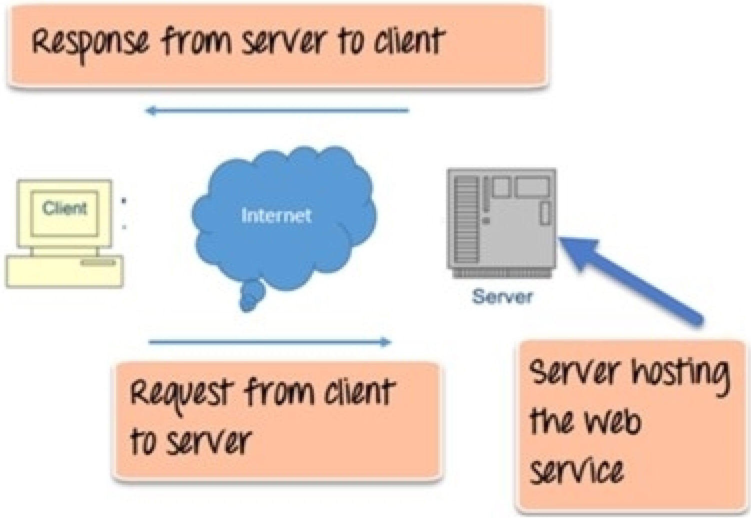
- Web services
- App Engine
- Azures Platform
- Aneka

Cloud Applications

- Scientific applications
- Business and Consumer applications

Web Services in Cloud Computing

- A web service is a standardized method for propagating messages between client and server applications on the World Wide Web.
- It is a set of open protocols and standards that allow data exchange between different applications or systems.
- A web service is a software module that aims to accomplish a specific set of tasks.
- Web services can be found and implemented over a network in cloud computing.
- The web service would be able to provide the functionality to the client that invoked the web service.
- The client would invoke a series of web service calls via requests to a server which would host the actual web service.
- These requests are made through what is known as remote procedure calls. Remote Procedure Calls(RPC) are calls made to methods which are hosted by the relevant web service.



Web Services in Cloud Computing

- The main component of a web service design is the data which is transferred between the client and the server, and that is XML.
- XML (Extensible markup language) is a counterpart to HTML and easy to understand the intermediate language that is understood by many programming languages.
- So when applications talk to each other, they actually talk in XML.
- This provides a common platform for application developed in various programming languages to talk to each other.

Web Service Components

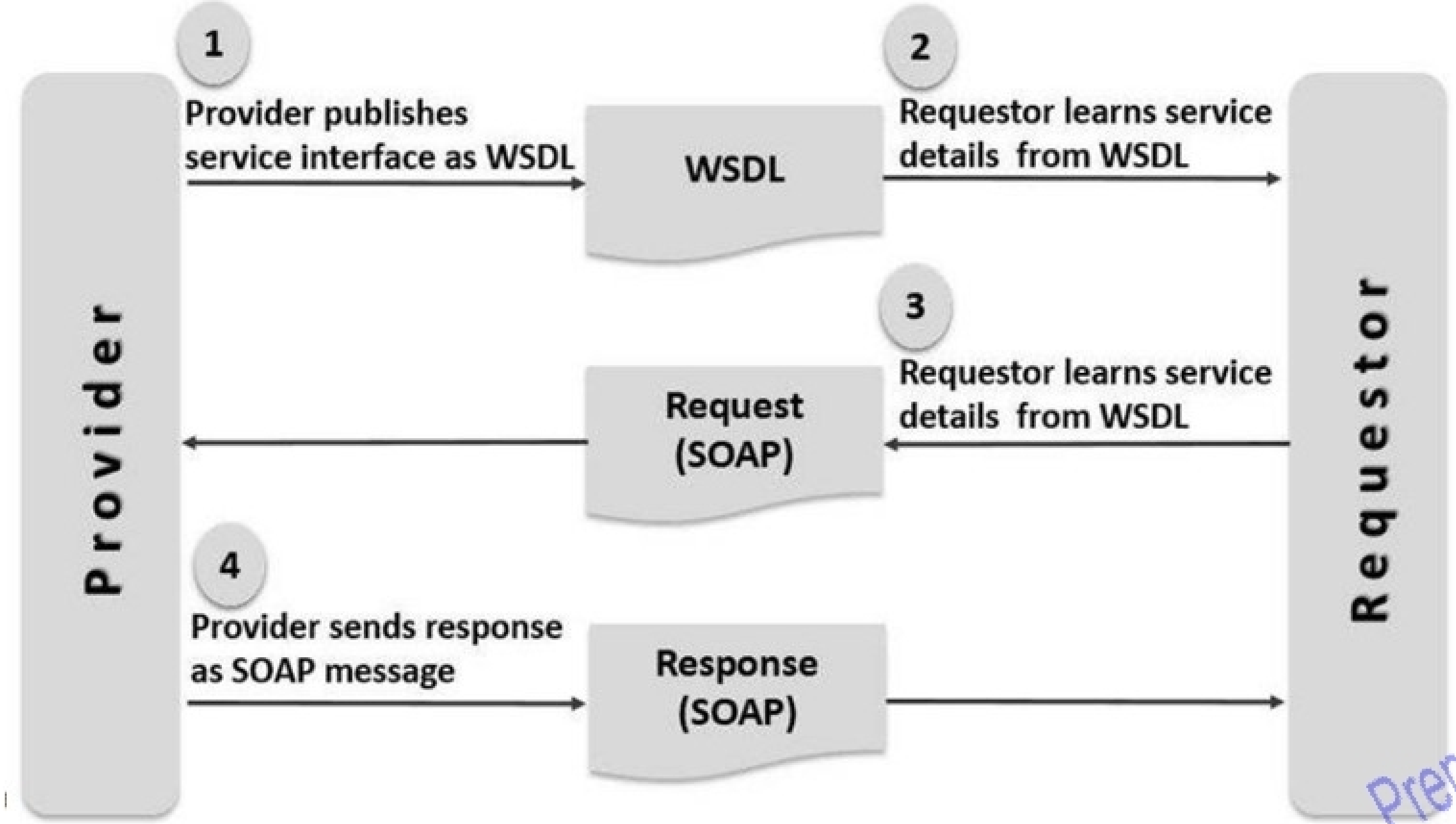
- XML and HTTP is the most fundamental web service platform.
- All typical web services use the following components:

1. SOAP (Simple Object Access Protocol) It is an XML-based messaging protocol for exchanging information among computers
2. UDDI (Universal Description, Search, and Integration) : UDDI is an XML-based standard for describing, publishing, and finding web services.
3. WSDL (Web Services Description Language) : WSDL is an XML-based protocol for information exchange in decentralized and distributed environments developed by IBM.

SOAP (Simple Object Access Protocol)

- SOAP stands for "Simple Object Access Protocol".
- It is a transport-independent messaging protocol.
- SOAP is built on sending XML data in the form of SOAP messages.
- A document known as an XML document is attached to each message.
- Only the structure of an XML document, not the content, follows a pattern.
- The great thing about web services and SOAP is that everything is sent through HTTP.
- Every SOAP document requires a root element known as an element.
- In an XML document, the root element is the first element.
- The "envelope" is divided into two halves. The header comes first, followed by the body.
- Routing data, or information that directs the XML document to which client it should be sent, is contained in the header while the real message will be in the body.

SOAP (Simple Object Access Protocol)



SOAP (Simple Object Access Protocol)

SOAP (Simple Object Access Protocol)

Request

```
POST /InStock HTTP/1.1
Host: www.example.org
Content-Type: application/soap+xml; charset=utf-8
Content-Length: nnn

<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">

<soap:Body xmlns:m="http://www.example.org/stock">
  <m:GetStockPrice>
    <m:StockName>IBM</m:StockName>
  </m:GetStockPrice>
</soap:Body>

</soap:Envelope>
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/soap+xml; charset=utf-8
Content-Length: nnn

<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">

<soap:Body xmlns:m="http://www.example.org/stock">
  <m:GetStockPriceResponse>
    <m:Price>34.5</m:Price>
  </m:GetStockPriceResponse>
</soap:Body>

</soap:Envelope>
```

UDDI (Universal Description, Search, and Integration)

- UDDI is a standard for specifying, publishing and searching online service providers.
- It provides a specification that helps in hosting the data through web services.
- UDDI provides a repository where WSDL files can be hosted so that a client application can search the WSDL file to learn about the various actions provided by the web service.
- As a result, the client application will have full access to UDDI, which acts as the database for all WSDL files.
- The UDDI Registry will keep the information needed for online services, such as a telephone directory containing the name, address, and phone number of a certain person so that client applications can find where it is.
- The client implementing the web service must be aware of the location of the web service.
- If a web service cannot be found, it cannot be used.
- Second, the client application must understand what the web service does to implement the correct web service.
- WSDL, or Web Service Description Language, is used to accomplish this.
- A WSDL file is another XML-based file that describes what a web service does with a client application.
- The client application will understand where the web service is located and how to access it using the WSDL document

Features of Web Service

- **XML-based:** A web service's information representation and record transport layers employ XML. There is no need for networking, operating system, or platform bindings when using XML.
- **Loosely Coupled**: The subscriber of an ISP may not necessarily be directly connected to that service provider. The user interface for a web service provider may change over time without affecting the user's ability to interact with the service provider. A loosely connected architecture makes software systems more manageable and easier to integrate between different structures.
- Ability to be synchronous or asynchronous: Synchronicity refers to the client's connection to the execution of the function.
- Asynchronous operations allow the client to initiate a task and continue with other tasks. The client is blocked, and the client must wait for the service to complete its operation before continuing in synchronous invocation.
- Asynchronous clients get their results later, but synchronous clients get their effect immediately when the service is complete.
- The ability to enable loosely connected systems requires asynchronous capabilities

Features of Web Service

- **Coarse Grain**: Object-oriented systems, such as Java, make their services available differently. At the corporate level, an operation is too great for a character technique to be useful.
- Building a Java application from the ground up requires the development of several granular strategies, which are then combined into a coarse grain provider that is consumed by the buyer or service.
- Corporations should be coarse-grained, as should the interfaces they expose. Building web services is an easy way to define coarse-grained services that have access to substantial business enterprise logic.
- Supports remote procedural calls: Consumers can use XML-based protocols to call procedures, functions, and methods on remote objects that use web services. A web service must support the input and output framework of the remote system.
- Supports document exchanges: One of the most attractive features of XML for communicating with data and complex entities.

Google Cloud Services

- Compute Engine – IaaS
- App Engine – PaaS
- Storage
  - Cloud SQL – Fully managed Relational MySQL
  - Cloud Storage – Object storage for applications
  - Cloud Datastore – Automatically scaled NoSQL storage
- Big Data
- Big Query – SQL like queries against multi-terabyte datasets

Google App Engine

- **Google App Engine** is an application hosting and development platform that powers everything from enterprise web applications to mobile games, using the same infrastructure that powers Google's global-scale web applications.
- It was announced on April 2008.
- Since then the objective of this platform has been to enable developers to run and develop their software somewhere in the Google cloud.
- Being able to do so, anyone may feel as if they were Google employees having access the entire scalable Google infrastructure.

https://cloud.google.com/appengine/

Google App Engine

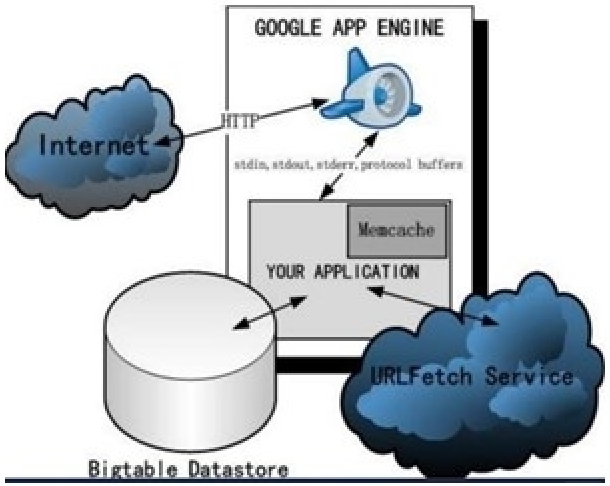
» One of the most important features of the Google App Engine is the fact that it is free for moderate levels of use.

» Every person that possesses a Gmail account can have a number of ten free applications running on the Google infrastructure and in case of one of them becoming very popular and the traffic going above the allowed levels of the free account, one can pay to use more of Google's resources.

» As the application scales, all the hardware, data storage, backup, and network provisioning for the customer are taken care of by Google´s engineers.

» The payment for Google´s resources is likely to be way lower than maintaining the same resources by the customer themselves.

» Google focuses on providing hardware and network, while the customer focuses on development and the user community around his application



Google App Engine

- A generalized view of GAE architecture, where datastore, services and internet connection via HTTP protocols are the factors on which the GAE application is based on. Supported languages:
- **Python Java Node.js Ruby C# Go PHP.**
- **Quick to start**: With no software or hardware to buy and maintain, you can prototype and deploy applications to your users in a matter of hours.
- **Simple to use**: Google App Engine includes the tools you need to create, test, launch, and update your apps.
- **Rich set of APIs**: Build feature-rich services faster with Google App Engine's easy-t-use APIs.
- **Immediate scalability**: There's almost no limit to how high or how quickly your app can scale.
- **Pay for what you use**: Get started without any upfront costs with App Engine's free tier and pay only for the resources you use as your application grows.

App Engine Characteristics

- **Multiple supporting languages**
- **Persistent storage** with queries, sorting, and transactions
- App Engine distributes user requests across multiple servers and scales servers to meet **dynamic traffic** demands.
- **Asynchronous task** queues for performing work outside the scope of a request
- **Scheduled tasks** for triggering events at specified times or regular intervals
- **Integration** with all other Google cloud services and APIs
- Your application runs within its **own secure, sandboxed** and reliable environment that is independent of the hardware, operating system, or physical location of the server.
- An SLA(**Service Level Agreement**) of 99.95% availability within a monthly billing cycle for paid applications using the High Replication Datastore

App Engine Integrated Services

- **App Identity**: provides an application with public key infrastructure managed by Google
- **Blobstore**: gives the application a possibility to store bigger data objects called BLOBs (Binary Large Objects) which are the objects that are much larger than the objects allowed in the Datastore service such as videos or images
- **Google Cloud Storage**: allows reading and writing files to Google Cloud Storage
- **Capabilities**: useful for detection of outages and scheduled downtime for particular API capabilities.
- **Channel**: This API enables to communicate directly with user browsers by pushing notifications directly to the JavaScript on the client-side, eliminating the need for polling
- **Conversion**: allows converting text to PDF and also has an option to perform optical character recognition (OCR)
- **Images:** provides the possibility of manipulating the image data and allows resizing, cropping, rotating or flipping images
- **Log Service**: provides access to the application logs, and request logs
- **Mail**: allows the application for sending email messages on behalf of the application's administrators and users with Google Accounts and also supports the JavaMail interface for sending email messages.
- **Memcache**: Provides caching ability with the basic services put(), get() and delete() for setting, fetching or deleting multiple items in a single batch call

App Engine Integrated Services

- **Multitenancy**: allows multiple client organizations/tenants to run the same instance of an application. Google achieves this functionality via Namespaces API, which allows for segregating data by using a unique namespace for each client
- **Oauth**: a protocol that enables a user to obtain a permit to access a web application in his name for a limited time without having to share his private data (username and password) with the third party
- **Prospective Search**: the website visitors are interested only in the data when newly entered records meet certain criteria
- **Search**: gives the application a possibility of Google-like searches over structured data within the application
- Task Queues: tool for background work. It allows for initiating a request that will make applications perform their work outside of that request.
- **URL Fetch**: allows Google App applications to communicate with other Internet hosts via HTTP and HTTPS requests.
- **Users**: the authentications of users with Google Accounts, accounts on their own Google Apps domain, or OpenID identifiers
- **XMPP**: allows applications to send and receive chat messages to and from any XMPPcompatible chat messaging service, such as Google Talk
- **Cron Service** : gives a possibility of task scheduling. Such tasks can be scheduled for execution at a defined time or every given time (regular intervals).

Windows Azure

- It is a platform-as-a-service(PaaS) offering from Microsoft
- It provides a strong full-featured platform for new cloud applications development, and migration of already existing enterprise application to the cloud.
- Windows Azure is a strictly public offering
- Microsoft claims that Azure offers a support for programming languages such as Java, node.js, Python, .net, PHP, although in order to develop in Java one would need to create a Virtual Machine, which is simply an IaaS offering not a PaaS.
- Moreover, PHP developers typically use development tools (such as Zend Framework) and runtime services (such as Apache, MySQL) that are not part of the Azure platform.
- Windows Azure is an operating system as a service – we can think of it as Windows in the cloud.
  - a. Azure is Microsoft's Cloud Computing offering for developers to build and deploy applications on a pay per use basis.
  - b. Azure is a comprehensively managed hosted platform for running applications and services.
  - c. The Windows Azure Platform is an internet-scale cloud services platform hosted in Microsoft data centers, which provides an operating system and a set of developer services that can be used individually or together.

Windows Azure

- Azure's flexible and interoperable platform can be used to build new applications to run from the cloud or enhance existing applications with cloud-based capabilities.
- Windows Azure provides a scalable infrastructure for customer to run their web services and applications.

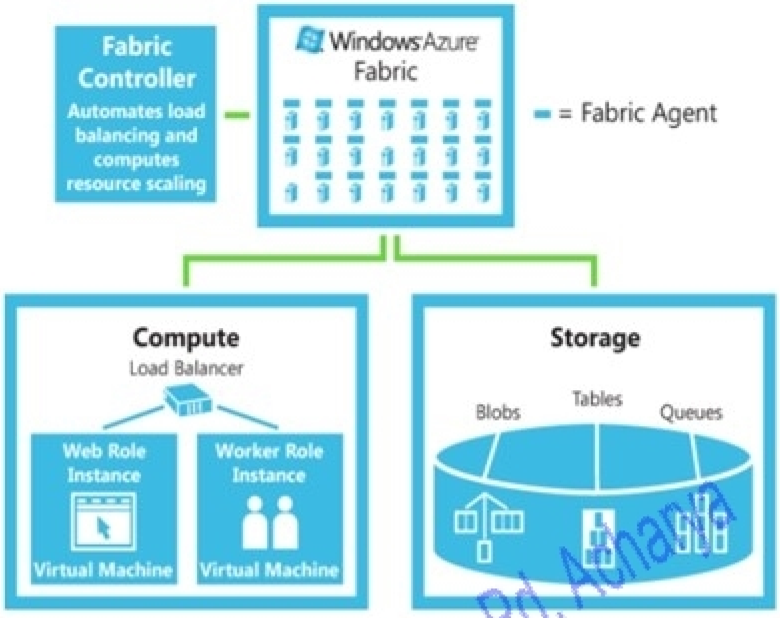
Programming language	.NET, PHP, Java, Ruby
Standards and protocols	SOAP, REST, and XML
Development tools	Visual Studio, Eclipse

Azure Support

- Azure system consists of a number of elements.
- 1. The Windows Azure Fabric Controller provides an auto-scaling and reliability and manages memory resources and load balancing.
- It provides two main areas of functionality:
- a) compute (e.g., executing an application)
- b) storage (e.g., storing data on disk)

Windows Azure

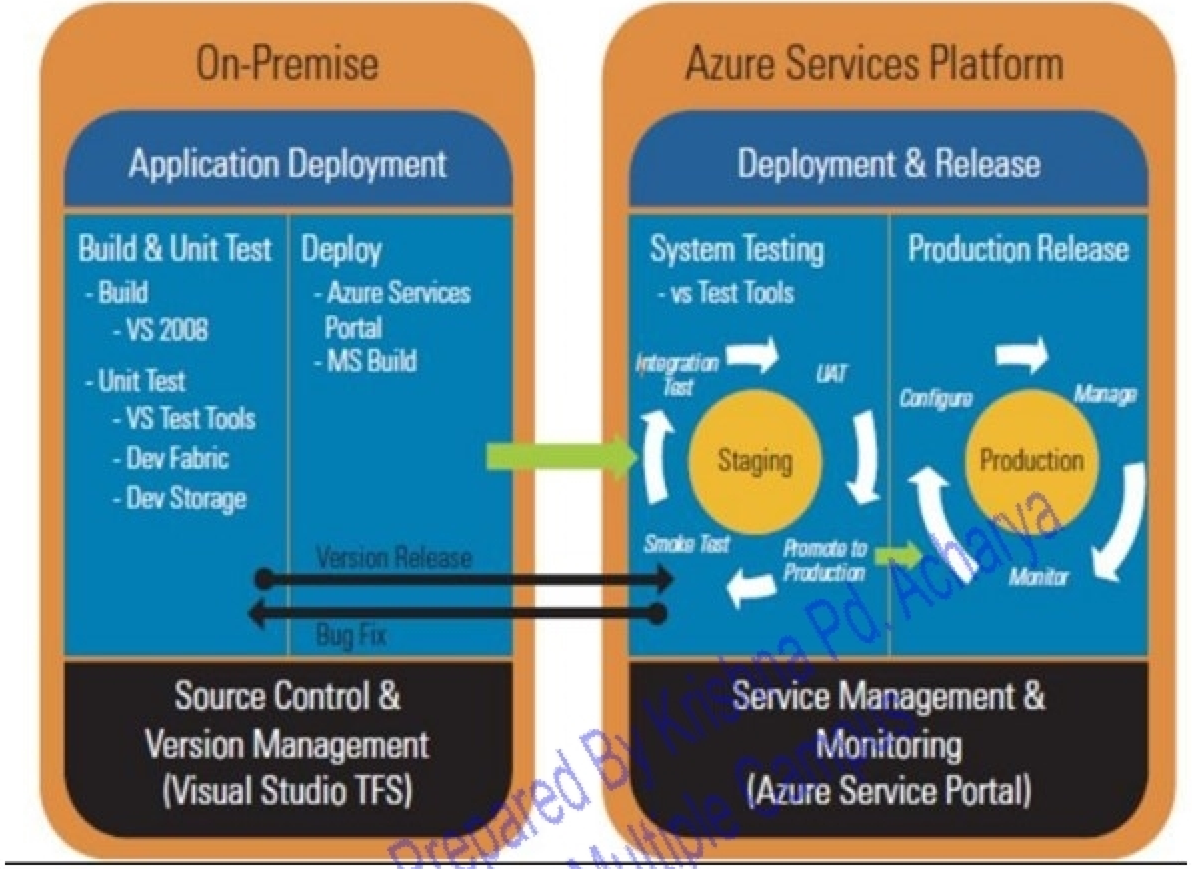
- The compute service offered by Windows Azure makes it possible to "execute" your applications in the cloud.
- The compute service provides a way to run your applications on a Windows Server running in a virtual machine hosted in Microsoft data center.
- Windows Azure storage services are designed to be very simple and highly scalable and allows storing for BLOB storage, queue storage, and simple table storage
- We interact with these storage services through a simple REST API based on HTTP requests and manipulate data in the storage services through traditional POST, PUT, and DELETE request
- The .NET Service Bus registers and connects applications together.
- 3. The .NET Access Control identity providers include enterprise directories and Windows LiveID.
- 4. Finally, the .NET Workflow allows construction and execution of workflow instances



Azure Platform

Azure Development Lifecycle

Azure development lifecycle will primarily follow two stages process.
1. Application Development:
In this stage, Azure application code is constructed locally on a developer's workstation
2. Deployment & Release:
This stage is carried out on the Azure development portal.
A user chooses to upload an application either in staging or production environments, along with the possibility for users to unchangeably swap the deployed code between the two environments.



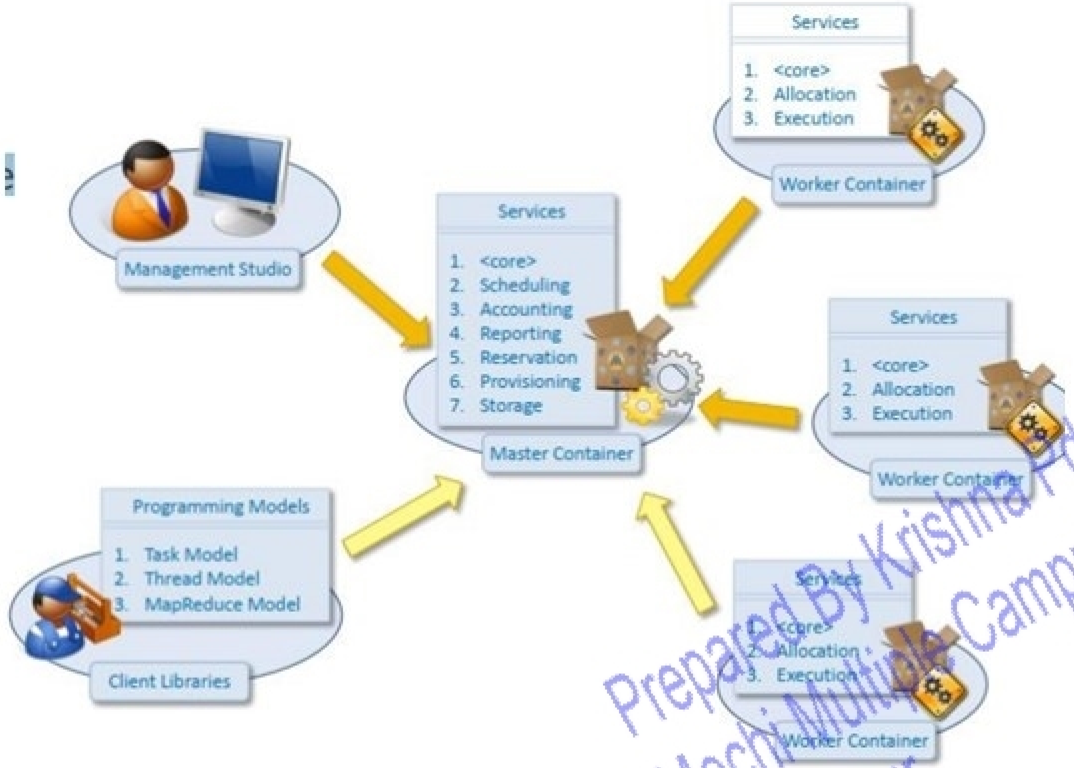
Aneka

- Aneka is a .NET-based application development Platform-as–a-Service (PaaS) for Cloud Computing(provided by Manjrasoft)
- It offers a runtime environment and a set of APIs that enable developers to build customized applications by using multiple programming models such as Task Programming, Thread Programming and MapReduce Programming
- Moreover, Aneka provides a number of services that allow users to control, auto-scale, reserve, monitor and bill users for the resources used by their applications.
- One of key characteristics of Aneka PaaS is to support provisioning of resources on public Clouds such as Windows Azure, Amazon EC2, and GoGrid, while also harnessing private Cloud resources ranging from desktops and clusters, to virtual data centres when needed to boost the performance of applications

Basic Architecture of Aneka

» The system includes four key components:

1. Aneka Master
2. Aneka Worker
3. Aneka Management Console
4. Aneka Client Libraries



Basic Architecture of Aneka

- The **Aneka Master** and **Aneka Worker** are both Aneka Containers which represents the basic deployment unit of Aneka based Clouds.
- The **Master Container** is responsible for managing the entire Aneka Cloud, coordinating the execution of applications by dispatching the collection of work units to the compute nodes
- The **Worker Container** is in charge of executing the work units, monitoring the execution, and collecting and forwarding the results
- The **Management Studio** and **client libraries** help in managing the Aneka Cloud and developing applications that utilize resources on Aneka Cloud.
- The **Management Studio** is an administrative console that is used to configure Aneka Clouds(install, start or stop Containers) setup user accounts and permissions for accessing Cloud resources; and access monitoring and billing information
- The **Aneka client libraries** are APIs used to develop applications which can be executed on the Aneka Cloud.

Business and Consumer applications

- Infrastructure as a service
- Platform as a service
- File storage
- Data backup
- Disaster recovery
- Increasing collaboration
- Testing new projects
- Quality control